KIỂM THỬ VÀ ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM
SỬ DỤNG CÔNG CỤ KIỂM THỬ ĐƠN VỊ



Giảng viên: Nguyễn Thị Phương Dung
Email: dungntp@tlu.edu.vn

Kiểm thử đơn vị

- Là phương pháp kiểm thử phần mềm trên từng modul nhỏ của chương trình.
- Trong các ngôn ngữ hướng đối tượng (ví dụ C#), các đơn vị kiểm thử này có thể là các hàm/ phương thức hay lớp.
- Một đơn vị kiểm thử được dùng để kiểm tra một đơn vị công việc với một kết quả giả định nào đó
- Nếu kết quả thực thi của đơn vị công việc cần kiểm tra khác với kết quả giả định thì đơn vị kiểm thử thất bại.



Một số đặc điểm của unit test:

- Code unit test phải ngắn gọn, dễ hiểu, dễ đọc.
- Mỗi unit test là 1 đơn vi riêng biệt, độc lập, không phụ thuộc vào unit khác.
- Mỗi unit test là 1 method trong test class, tên method cũng là tên UnitTest. Do đó ta nên đặt tên hàm rõ ràng, nói rõ unit test này test cái gì (Test_A_Do_B), tên method có thể rất dàiii cũng không sao.
- Unit Test phải nhanh, vì nó sẽ được chạy để kiểm định lỗi mỗi lần build. Do đó trong unit test nên hạn chế các task tốn thời gian như gọi I/O, database, network,…
- Unit Test nên test từng đối tượng riêng biệt. Vd: Unit Test cho Business Class thì chỉ test chính BusinessClass đó, không nên dụng tới các class móc nối với nó (DataAccess Class chẳng hạn).



Framework cho Unit Testing

- Unit Testing có thể được thực hiện dễ dàng nhờ các framework.
- Một số Framework phổ biến hỗ trợ cho các nhà phát triển .Net:
  - NUnit
  - xUnit
  - MSTest




Sử dụng NUnit

Các bước sử dụng NUnit

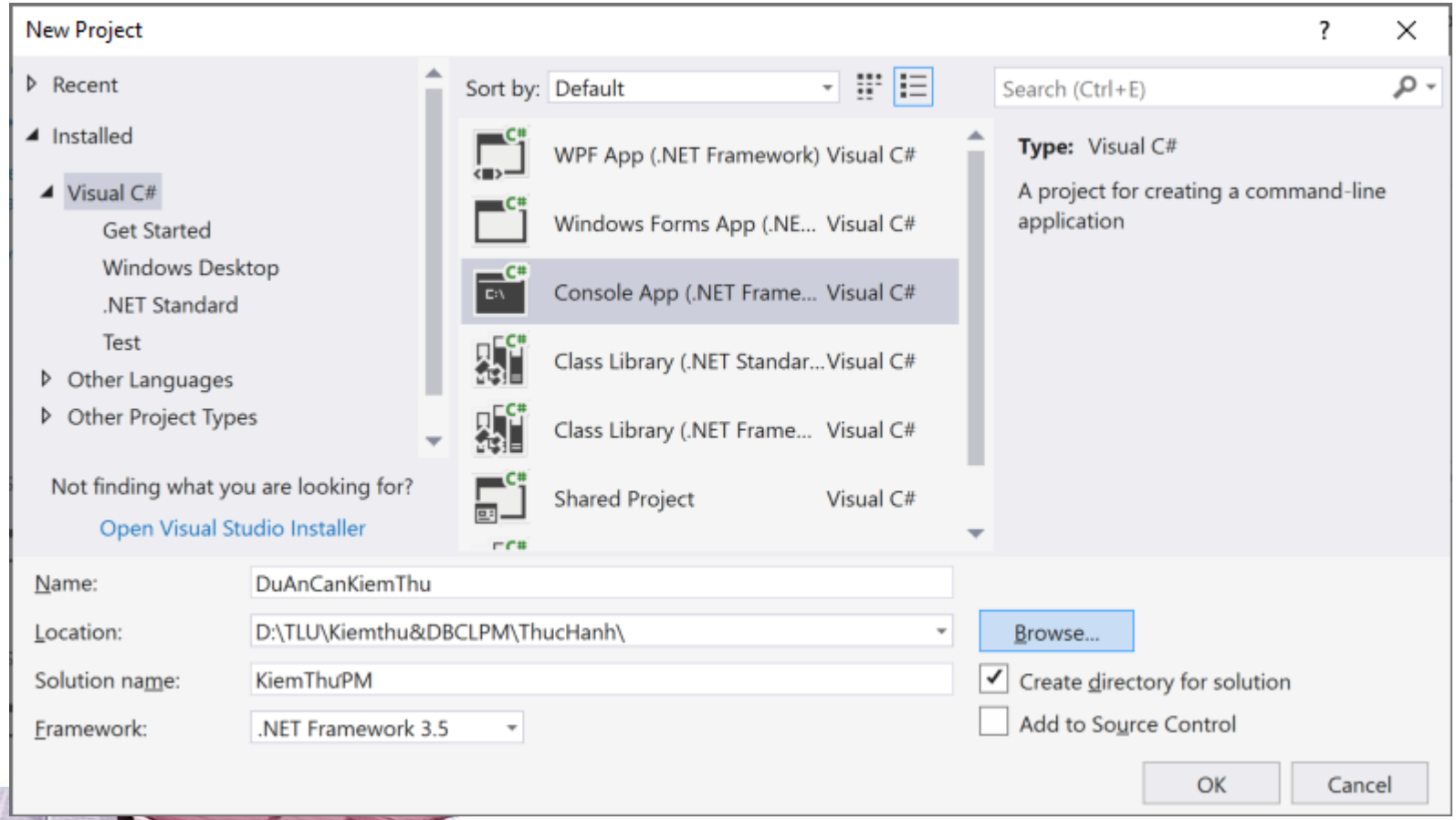
- Tạo dự án cần kiểm thử
- Tạo dự án kiểm thử
- Tạo liên kết giữa 2 dự án trên
- Cài đặt Nunit để được hỗ trợ framework test
- Cài đặt NUnit3TestAdapter để sử dụng các phương thức thực thi các đơn vị kiểm thử.
- Thực hiện viết lệnh
- Thực hiện test



Tạo dự án cần kiểm thử

- Sử dụng Visual Studio tạo một solution
- Trong solution tạo một project cần kiểm thử
- Trong project tạo các hàm cần kiểm thử, đây chính là các đơn vị cần kiểm thử

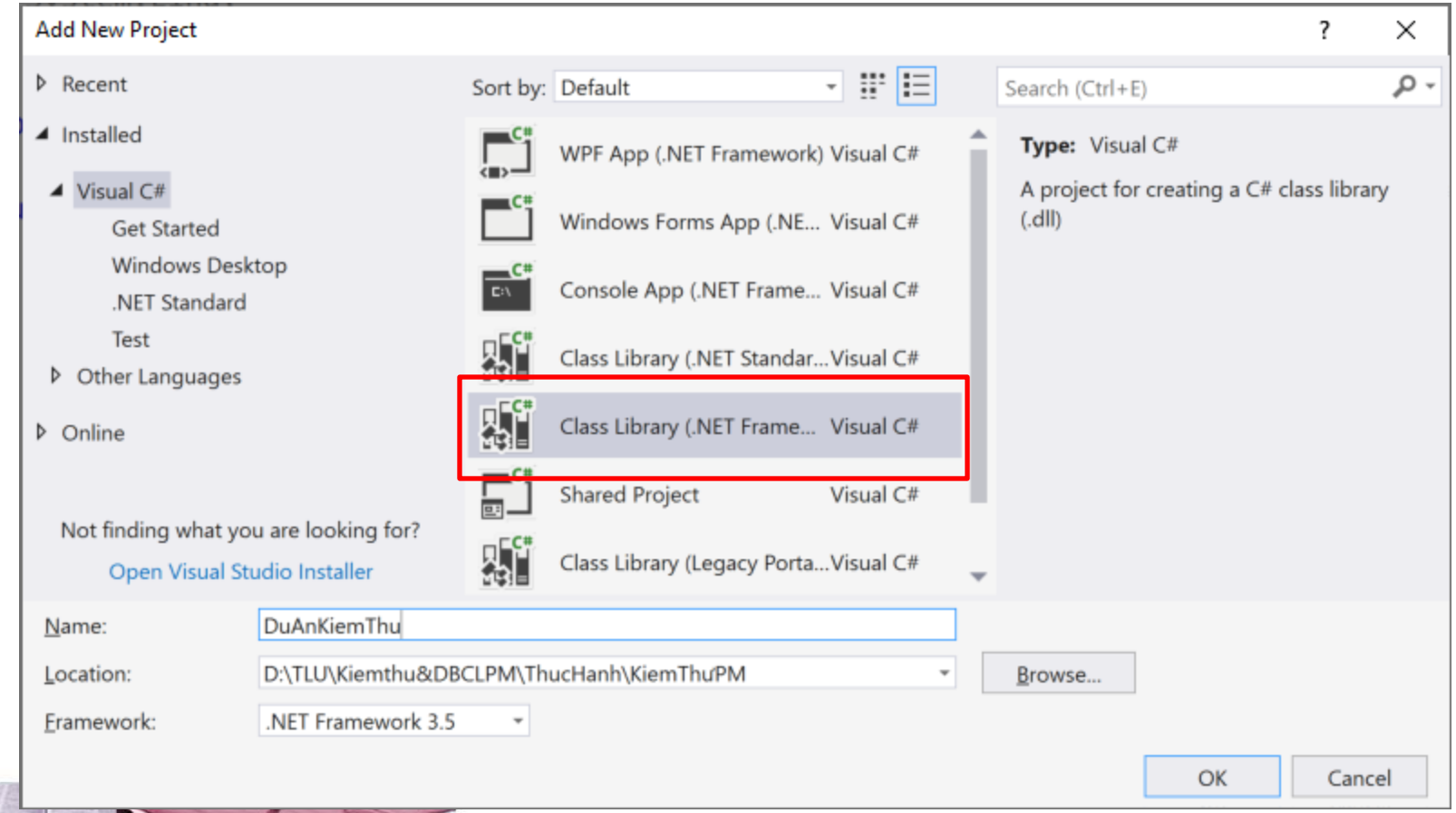



New Project
Recent
Installed
Visual C#
Get Started
Windows Desktop
.NET Standard
Test
Other Languages
Other Project Types
Not finding what you are looking for?
Open Visual Studio Installer
Sort by: Default
Search (Ctrl+E)
WPF App (.NET Framework) Visual C#
Windows Forms App (.NE... Visual C#
Console App (.NET Frame... Visual C#
Class Library (.NET Standar... Visual C#
Class Library (.NET Frame... Visual C#
Shared Project Visual C#
Type: Visual C#
A project for creating a command-line application
Name: DuAnCanKiemThu
Location: D:\TLU\Kiemthu&DBCLPM\ThucHanh\
Solution name: KiemThuPM
Framework: .NET Framework 3.5
Browse...
Create directory for solution
Add to Source Control
OK
Cancel

Tạo dự án kiểm thử

- Tạo project kiểm thử đơn vị trong cùng solution với project cần kiểm thử
- Kiểu của project là Class Library

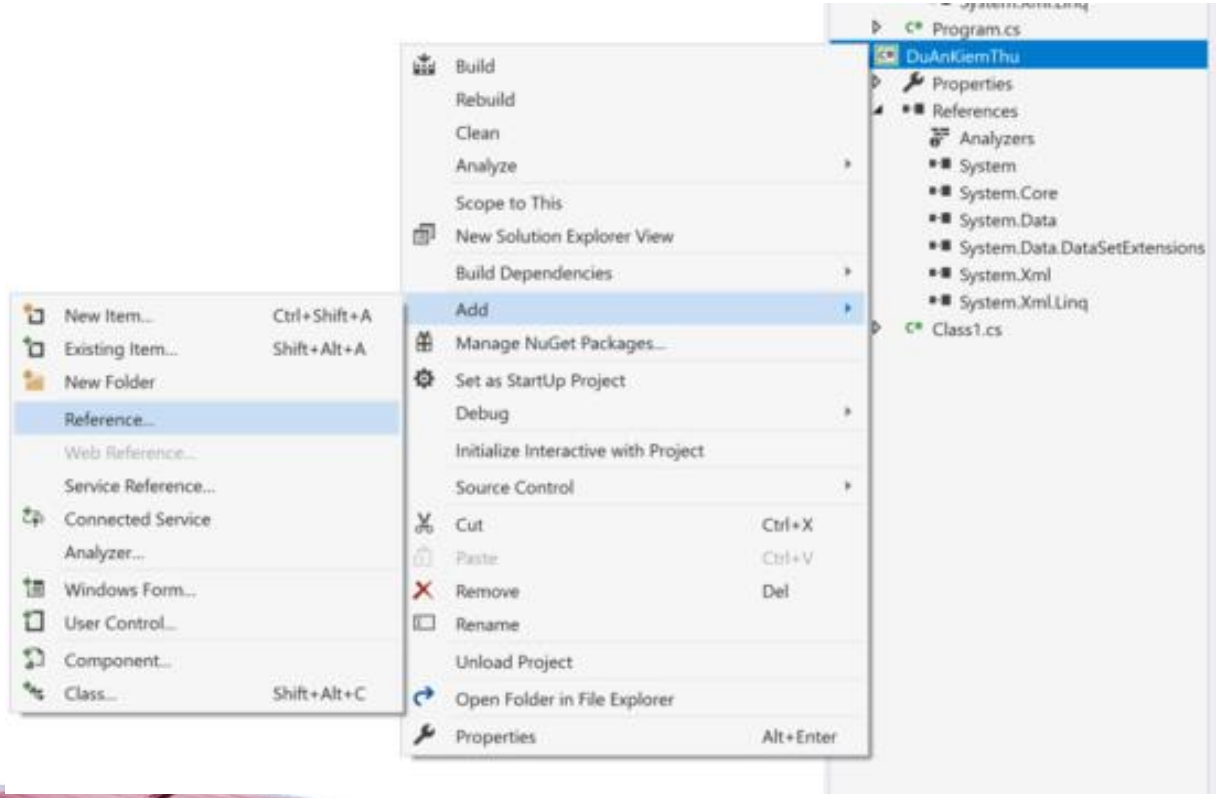


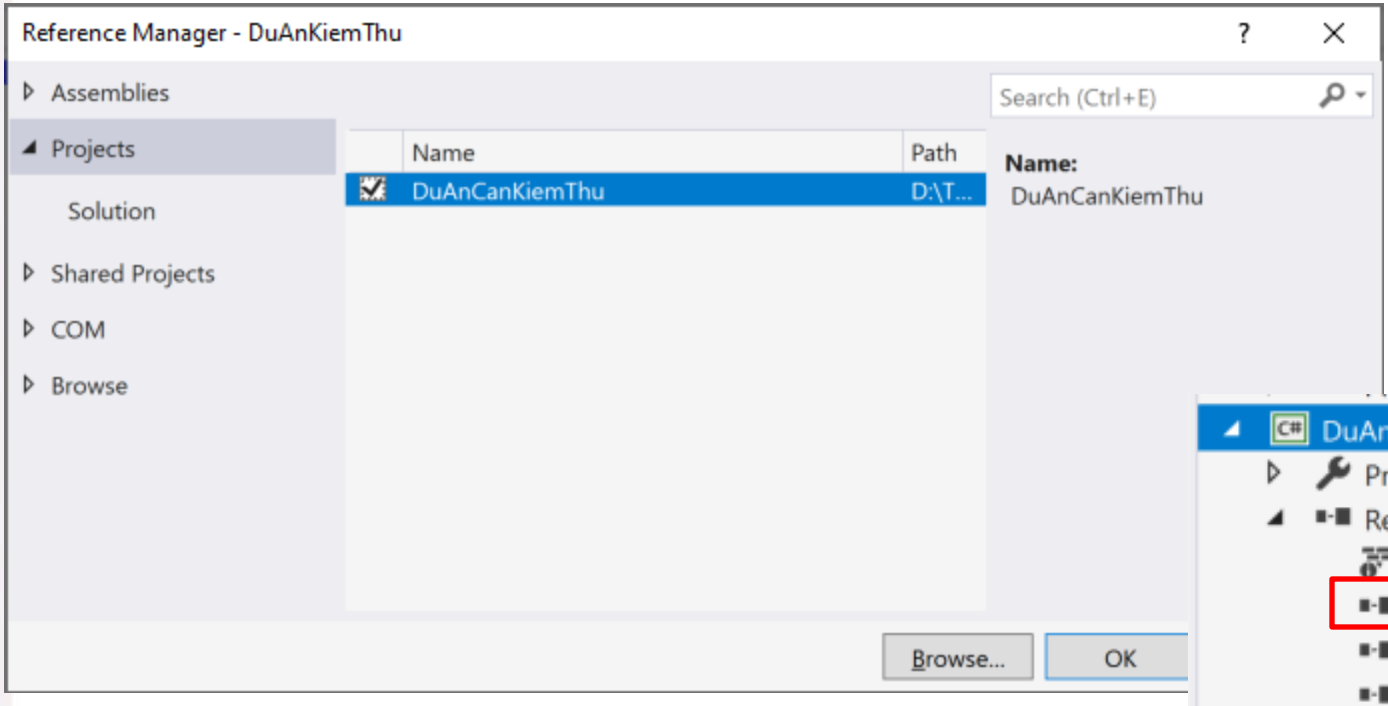

Add New Project
Recent
Installed
Visual C#
Get Started
Windows Desktop
.NET Standard
Test
Other Languages
Online
Not finding what you are looking for?
Open Visual Studio Installer
Sort by: Default
Search (Ctrl+E)
WPF App (.NET Framework) Visual C#
Windows Forms App (.NE... Visual C#
Console App (.NET Frame... Visual C#
Class Library (.NET Standar... Visual C#
Class Library (.NET Frame... Visual C#
Shared Project Visual C#
Class Library (Legacy Porta... Visual C#
Type: Visual C#
A project for creating a C# class library (.dll)
Name: DuAnKiemThu
Location: D:\TLU\Kiemthu&DBCLPM\ThucHanh\KiemThuPM
Framework: .NET Framework 3.5
Browse...
OK
Cancel

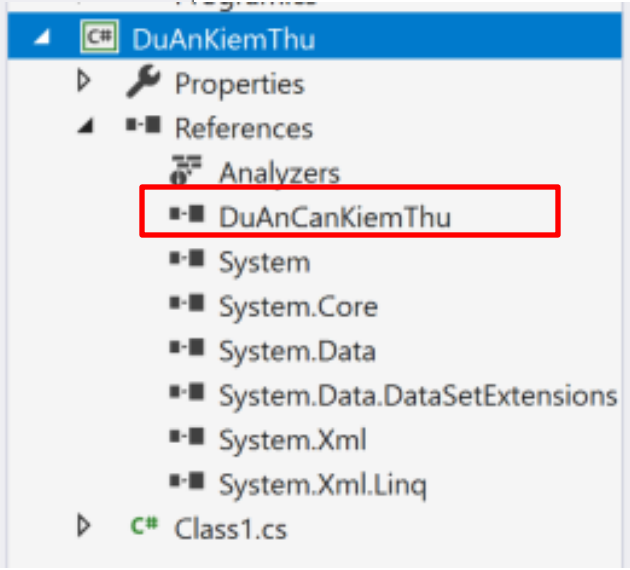
Tạo liên kết giữa 2 dự án

- Trong cửa sổ solution, nhấn chuột phải vào dự án kiểm thử, chọn Add > Reference.
- Trong cửa sổ Reference Manager chọn dự án cần kiểm thử




Program.cs
DuAnKiemThu
Properties
References
Analyzers
System
System.Core
System.Data
System.Data.DataSetExtensions
System.Xml
System.Xml.Linq
Class1.cs
Build
Rebuild
Clean
Analyze
Scope to This
New Solution Explorer View
Build Dependencies
Add
Manage NuGet Packages...
Set as StartUp Project
Debug
Initialize Interactive with Project
Source Control
Cut Ctrl+X
Paste Ctrl+V
Remove Del
Rename
Unload Project
Open Folder in File Explorer
Properties Alt+Enter
New Item... Ctrl+Shift+A
Existing Item... Shift+Alt+A
New Folder
Reference...
Web Reference...
Service Reference...
Connected Service
Analyzer...
Windows Form...
User Control...
Component...
Class... Shift+Alt+C

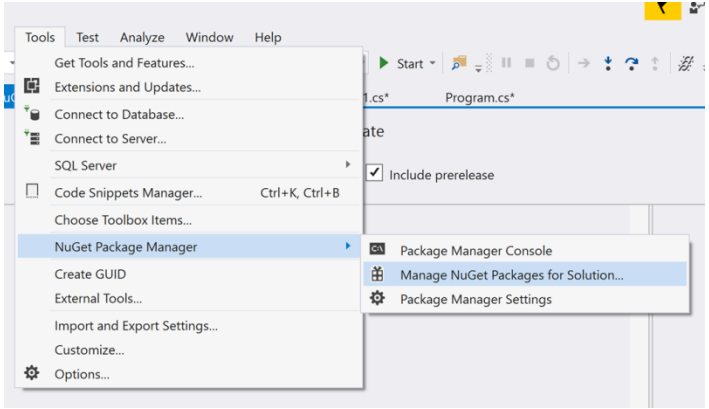

Reference Manager - DuAnKiemThu
Assemblies
Projects
Solution
Shared Projects
COM
Browse
Search (Ctrl+E)
Name
Path
DuAnCanKiemThu
D:\T...
Name:
DuAnCanKiemThu
Browse...
OK


DuAnKiemThu
Properties
References
Analyzers
DuAnCanKiemThu
System
System.Core
System.Data
System.Data.DataSetExtensions
System.Xml
System.Xml.Linq
Class1.cs



Cài đặt NUnit

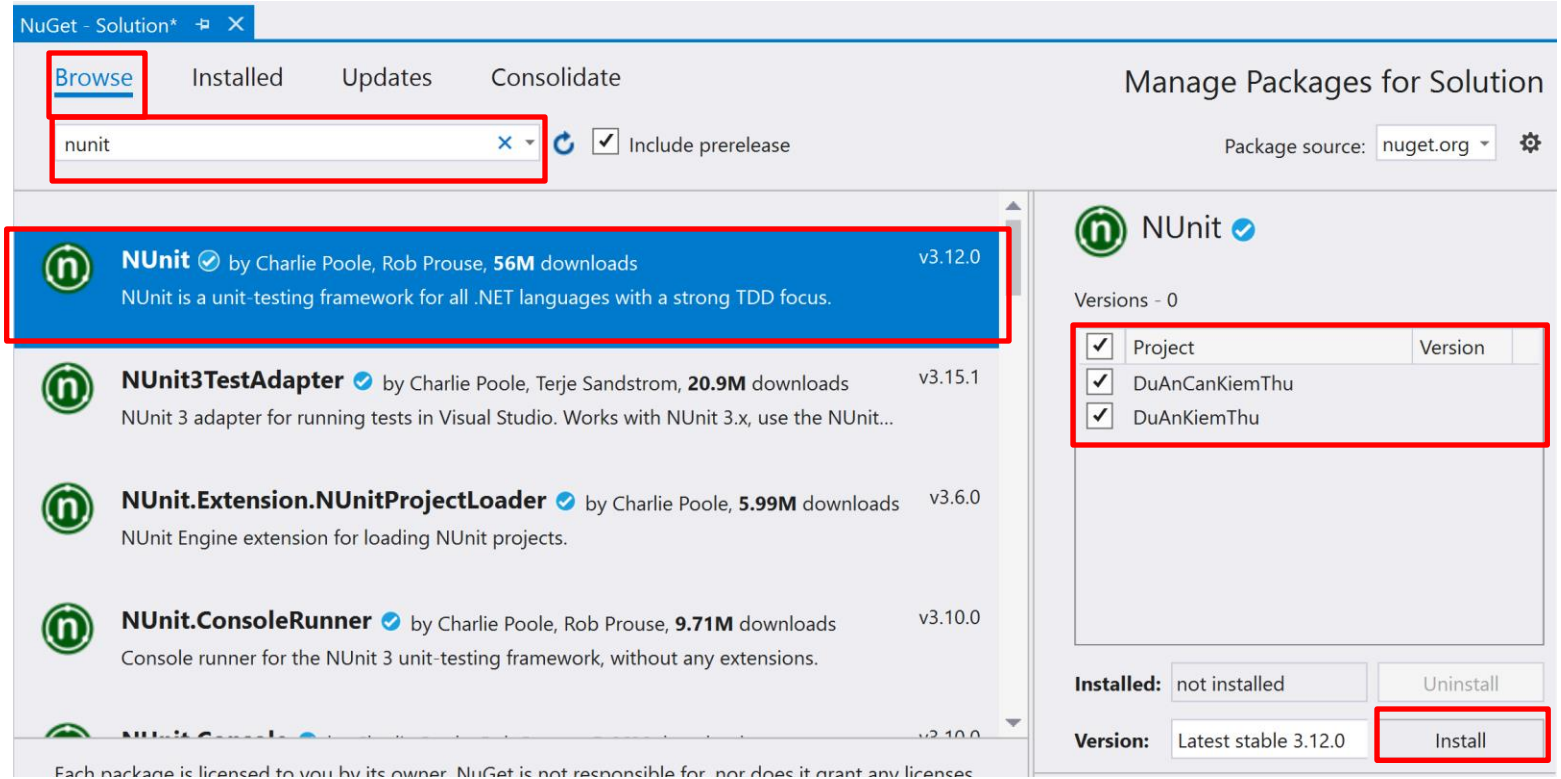
- Chọn Tool/NuGet Package Manager/Manage NuGet Packages for Solution



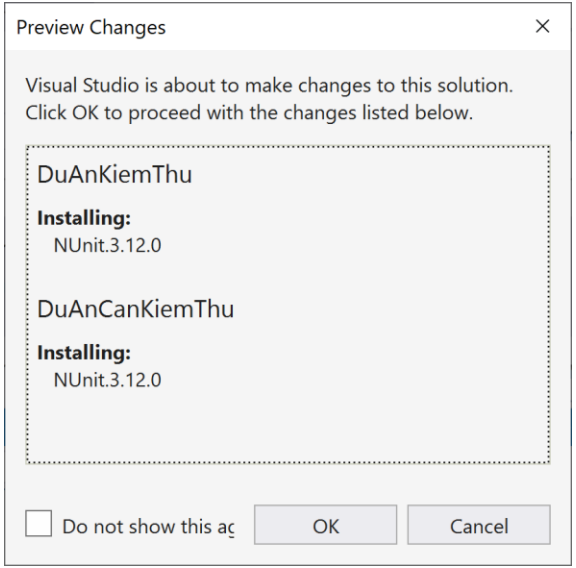
Cài đặt NUnit

- Trong cửa sổ NuGet – Solution chọn tab Browse và gõ cụm từ '*nunit*' trong ô tìm kiếm và chọn Nunit
- Bên khung cửa sổ NUnit bên phải chọn Project (bao gồm hai dự án) và nhấn Install:




NuGet - Solution*
Browse
Installed
Updates
Consolidate
Manage Packages for Solution
nunit
Include prerelease
Package source:
nuget.org
NUnit by Charlie Poole, Rob Prouse, 56M downloads
v3.12.0
NUnit is a unit-testing framework for all .NET languages with a strong TDD focus.
NUnit3TestAdapter by Charlie Poole, Terje Sandstrom, 20.9M downloads
v3.15.1
NUnit 3 adapter for running tests in Visual Studio. Works with NUnit 3.x, use the NUnit...
NUnit.Extension.NUnitProjectLoader by Charlie Poole, 5.99M downloads
v3.6.0
NUnit Engine extension for loading NUnit projects.
NUnit.ConsoleRunner by Charlie Poole, Rob Prouse, 9.71M downloads
v3.10.0
Console runner for the NUnit 3 unit-testing framework, without any extensions.
NUnit
Versions - 0
Project
Version
DuAnCanKiemThu
DuAnKiemThu
Installed:
not installed
Uninstall
Version:
Latest stable 3.12.0
Install

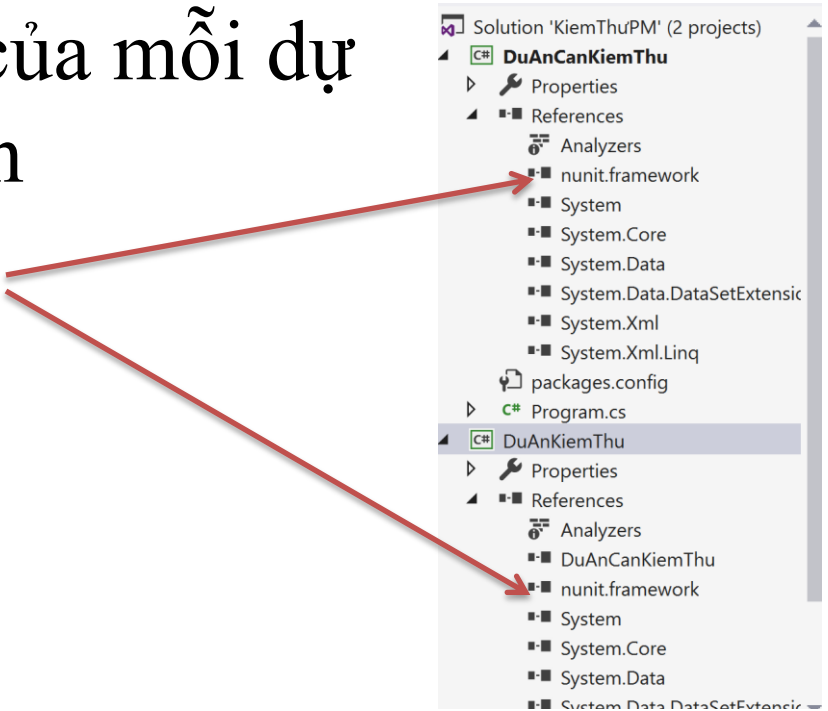
- Nếu xuất hiện cửa sổ preview changes thì ấn nút OK





Kiểm tra cài đặt NUnit

- Trong References của mỗi dự án sẽ thấy xuất hiện nunit.framework





Cài đặt NUnit3TestAdapter

- Mục đích: để sử dụng các phương thức thực thi các đơn vị kiểm thử NUnit trong Visual Studio.



Các thuộc tính TestFixture, Test

- ***NUnit*** chứa hai thuộc tính ***TestFixture*** và ***Test*** dùng để đánh dấu lớp và phương thức sẽ là các đơn vị kiểm thử (unit tests).
- Trước khi sử dụng hai thuộc tính này, cần khai báo thư viện ***Nunit***

```
using NUnit.Framework;
```



Các thuộc tính TestFixture, Test

- Thuộc tính TestFixture được đặt trong cặp dấu ngoặc vuông [] trước tên lớp kiểm thử
- Thuộc tính Test được đặt trong cặp dấu ngoặc vuông [] trước tên phương thức kiểm thử



Các thuộc tính TestFixture, Test

```
using NUnit.Framework;

namespace DuAnKiemThu
{
    [TestFixture]
    public class KiemThu
    {
        [Test]
        public void Test_KtraNT()
        {

        }
        [Test]
        public void Test_Nhapso()
        {

        }
    }
}
```



Lớp Assert

- Lớp Assert như chiếc cầu nối giữa mã chương trình cần kiểm thử và NUnit
- Mỗi đơn vị kiểm thử sẽ dùng lớp này với mục đích khai báo một giả định nào đó là tồn tại.
- Nếu các đối số được chuyển vào lớp ***Assert*** thực hiện cho ra kết quả khác với giả định thì đơn vị kiểm thử này tìm ra lỗi, ngược lại chương trình không bị lỗi.



Các phương thức của lớp Assert

Phương thức	Mô tả
Assert.AreEqual(expected, actual [, string message])	Là phương thức thường xuyên được dùng trong quá trình kiểm thử. expected là giá trị kỳ vọng; actual là giá trị thực; message là thông điệp sẽ hiển thị nếu kiểm thử thất bại.
Assert.Less(x, y)	Xác nhận x < y
Assert.Greater(x,y)	Xác nhận x > y



Các phương thức của lớp Assert

Phương thức	Mô tả
Assert.GreaterOrEqual(x, y) *Assert.LessOrEqual(x,y)*	Xác nhận x <= y Xác nhận x >= y
Assert.IsNull(object [, string message]) *Assert.IsNotNull(object [, string message])*	Xác nhận một đối tượng là null, ngược lại kiểm thử thất bại. Xác nhận một đối tượng là khác null, ngược lại kiểm thử thất bại.



Các phương thức của lớp Assert

Phương thức	Mô tả
Assert.IsTrue(bool condition [, string message])	Xác nhận điều kiện đã cho là đúng, ngược lại kiểm thử thất bại.
Assert.IsFalse(bool condition [, string message])	Xác nhận điều kiện đã cho là sai, ngược lại kiểm thử thất bại.



Ví dụ

```
using System.Text;

namespace DuAnCanKiemThu
{
    public class KiemTraSo
    {
        public bool KtraSoNT(int x)
        {
            int a;
            for(int i = 2; i<=x/2; i++)
                if(x%i == 0)
                    return false;
            return true;
        }
        public int cong(int a, int b)
        {
            return a - b;
        }
    }
}
```

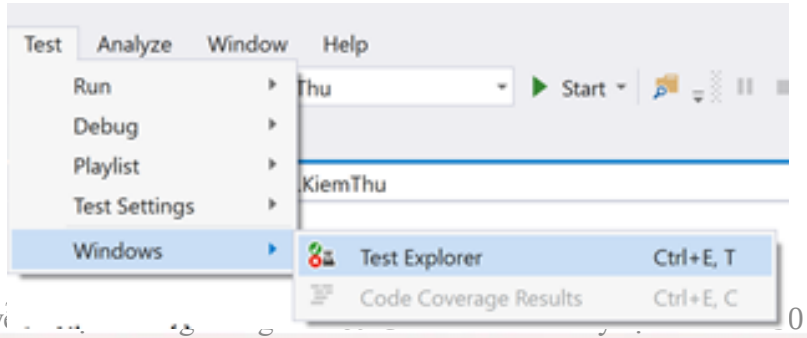
```
using NUnit.Framework;
using DuAnCanKiemThu;

namespace DuAnKiemThu
{
    [TestFixture]
    public class KiemThu
    {
        KiemTraSo kt = new KiemTraSo();
        [Test]
        public void Test_KtraNT()
        {
            bool x = kt.KtraSoNT(1);
            Assert.IsFalse(x);
        }
        [Test]
        public void Test_Cong()
        {
            Assert.AreEqual(kt.cong(8, 9), 17);
        }
    }
}
```



Thực hiện kiểm thử

- Sử dụng cửa sổ Test Explorer để thực thi các unit test
- Mở cửa sổ Test Explorer bằng cách vào menu Test/Windows/Test Explorer





Thực hiện kiểm thử

- Tại cửa sổ Test Explorer
  - Bấm chuột phải vào đơn vị test cần chạy
  - Chọn Run Selected Tests
  - Kết quả sẽ được hiển thị ở bên dưới cửa sổ Test Explorer

